시스템 분석 및 설계

06주차. 기본 설계(2)

서울사이버대학교

학습목표

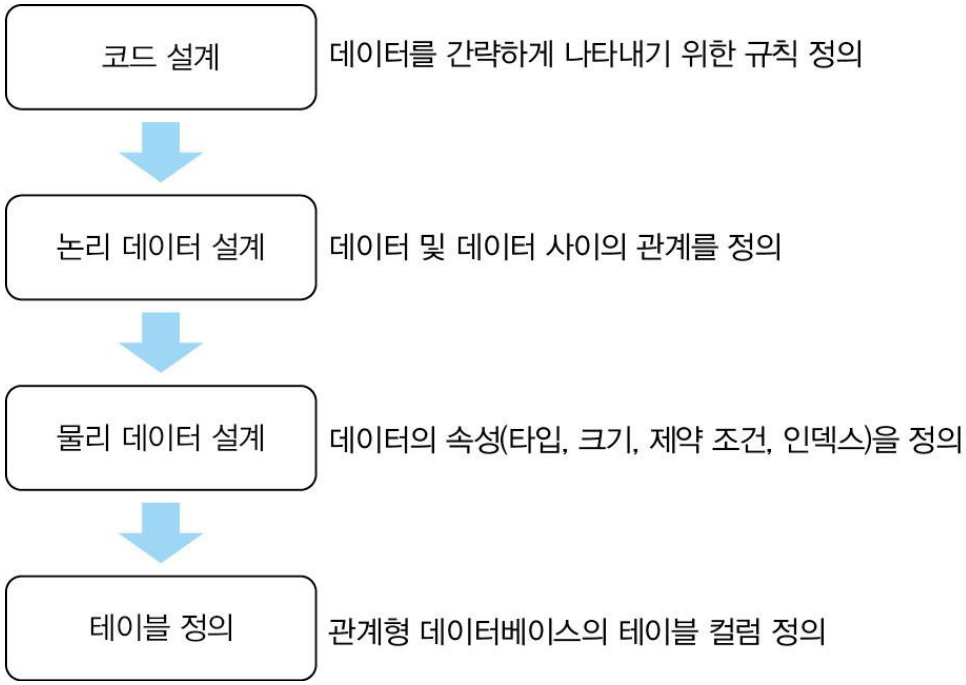
1. 다양한 코드화 방법의 의미와 사용방법을 이해한다.
2. ER다이어그램 작성 방법의 개요를 이해한다
3. CRUD 매트릭스를 작성할 수 있다.
4. 기본 설계서의 개념을 이해하고 항목에 들어갈 요소를 이해한다.

데이터 설계는 일반적으로 소프트웨어 시스템에서 사용하는 데이터를 찾아내 파일에 저장하는 항목의 요소나 데이터베이스에 저장할 테이블의 구조를 디자인하는 작업

- 파일이나 데이터베이스는 소프트웨어의 중요한 구성요소로 인터페이스와 상호 작용하며 자료를 저장하는 곳
- 따라서 자료 저장소가 어떤 항목을 어떻게 저장하고 관리할지 정의하는 것이 자료 설계
- 나중에 처리라는 요소가 어떻게 이뤄질지, 사용자 인터페이스와 연결시켜 고민해야 함

정보 시스템은 업무와 관련된 여러가지 자료를 처리하여 정보를 제공하는 역할을 담당

- 데이터베이스는 기업이 가지고 있는 자산, 직원 정보, 영업 정보, 거래처 등 경영활동과 기업 존재에 대한 모든 정보가 들어 있음
- 따라서 기업 시스템의 핵심은 바로 이러한 데이터를 어떻게 잘 관리하고 활용하느냐에 달려 있음



자료 설계 과정

시스템이 취급하는 정보 중 표시하는 방법이 여러가지이며, 입력에 따라 자료 값에 편차가 생길 수 있는 정보는 코드화
주민등록 번호, 학번, 우편번호와 같은 정보를 코드화 하면 효과적으로 잘 처리할 수 있음

코드화는 데이터에 대하 번호 또는 영숫자 기호로 만든 특정 코드를 할당하는 것을 의미함

- 가령 고객의 성별, 주소, 상품명 등은 코드화의 대상이 되는 경우가 많음
- 소프트웨어 시스템을 구현할 때 반복되는 자료 값이나, 반복되지 않더라도 입력과 저장의 효율을 위해 기본 설계 단계에서 자료 값과 코드의 대응을 정의하여 코드화 된 값을 사용함

1.순차 코드 (Sequential Code)

•이름: 일련번호 코드, 연속 코드

•설명: 데이터가 발생하거나 입력되는 순서대로 일련번호를 부여하는 방식

•예시

- 도서관 도서번호: B0001, B0002, B0003
- 회원번호: M2024001, M2024002
- 은행 대기번호: A01, A02, A03
- 제품 시리얼넘버: SN0001, SN0002

•장점

- 구현이 매우 단순
- 중복 발생 위험 없음
- 자동 번호 부여 용이
- 데이터 정렬이 쉬움

•단점

- 데이터의 특성이나 의미 파악 불가
- 중간 번호 삭제 시 재사용 여부 결정 필요
- 대량 데이터 처리 시 번호 체계 관리 복잡

2.구분 코드 (Block Code)

•이름: 블록 코드, 범주 코드

•설명: 데이터의 특성이나 범주에 따라 구분하여 코드를 부여하는 방식

•예시

- 대학 학번: 공대(1xxxxx), 문과(2xxxxx), 의대(3xxxxx)
- 직원 사번: 임원(A-xxx), 관리자(B-xxx), 사원(C-xxx)
- 창고 물품 위치: A구역(A-xx-xx), B구역(B-xx-xx)
- 진료과별 차트번호: 내과(IM-xxxx), 외과(SG-xxxx)

•장점

- 데이터의 소속/특성을 즉시 파악 가능
- 분류별 통계/관리 용이
- 그룹별 데이터 처리 효율적

•단점

- 새로운 구분 추가 시 코드 체계 수정 어려움
- 복수 특성 보유 데이터의 분류 모호
- 각 구분별 충분한 번호 공간 필요

3.십진 코드 (Decimal Code)

• 계층 코드, 데시멀 코드

•설명: 0~9까지의 숫자를 사용하여 계층적 구조를 표현하는 방식

•예시

- 도서 분류: 000(총류), 100(철학), 110(형이상학)
- 조직도 코드: 1000(영업부), 1100(국내영업팀)
- 상품 분류: 1000(전자제품), 1100(TV), 1110(OLED TV)
- 지역 코드: 10(서울시), 11(종로구), 111(청운동)

•장점

- 계층 구조의 명확한 표현
- 각 계층별 확장 용이
- 상하위 분류 관계 파악 쉬움

•단점

- 계층 깊어질수록 코드 길이 증가
- 계층당 최대 10개 하위분류로 제한
- 다중 분류 체계 표현 어려움

4.유효 숫자 코드 (Significant Digit Code)

•의미 숫자 코드, 식별 숫자 코드

•설명: 각 자리의 숫자가 특정한 의미를 가지도록 설계하는 방식

•예시

- 제품코드: YYYYMMXXXXX(연도+월+일련번호)
- 학번: 2024110001(입학연도+학과+일련번호)
- 주민등록번호: YYMMDD-NNNNNNN
- 신용카드 번호: 첫 자리(3:AMEX, 4:VISA, 5:MC)

•장점

- 코드에 다양한 정보 포함 가능
- 데이터 특성과 생성 시점 파악 용이
- 체계적인 정보 관리 가능

•단점

- 코드 구조가 복잡
- 각 자리수 의미에 대한 이해 필요
- 포함 가능한 정보량 제한적

5.그룹 분류식 코드 (Group Classification Code)

•조합 코드, 다중 분류 코드

•설명: 여러 가지 특성을 조합하여 하나의 코드로 표현하는 방식

•예시

- 상품코드: AA-BB-CC-DDD(대분류-중분류-소분류-일련번호)
- 도면번호: 01-02-03-004(프로젝트-공정-설계분야-일련번호)
- 군수품 재고번호: 국가코드-분류번호-품목식별번호
- 의약품 표준코드: 제약회사-제품-포장단위-검증번호

•장점

- 다양한 특성의 체계적 표현
- 각 그룹별 분류/관리 용이
- 높은 코드 확장성

•단점

- 코드 길이가 길어짐
- 코드 체계 설계 복잡
- 그룹 간 경계가 모호할 수 있음

6.연상 코드 (Mnemonic Code)

•기억 코드, 약어 코드

•설명: 데이터의 특성을 연상할 수 있는 문자나 약어를 사용하는 방식

•예시

- 부서코드: ACC(회계), HR(인사), MKT(마케팅)
- 거래처코드: IBM, MSFT, APPL
- 공항코드: ICN, LAX, JFK
- 파일확장자: txt, doc, pdf, jpg

•장점

- 직관적 의미 이해 가능
- 기억하기 쉬움
- 사용자 친화적

•단점

- 코드 길이 불일정
- 언어/문화별 이해도 차이
- 동음이의어/약어 중복 가능성

논리 데이터 설계는 프로그램이 보관하고 유지하는 데이터와 데이터 사이의 관계와, 그리고 데이터베이스의 구조를 정의하기 위해 수행하는 작업

- 논리 데이터 설계에서는 데이터 구조와 관계를 그림으로 표현하고, 처리에 적합한 데이터 구조를 작성함
- 관계형 데이터 베이스에 있는 테이블에 있는 종류와 구조를 나타낸 데이터베이스의 목록 등도 기본 설계 단계에서 작성함
- 논리 데이터 설계에 이용하는 대표적인 다이어그램은 ER다이어그램(ERD)과 CRUD 매트릭스 등이 대표적

ER다이어그램은 실세계를 엔티티(Entitiy)로 파악하고, 엔티티 사이에 상호 관계를 관계(Relationship)로 표현한 그림

- 데이터베이스에서 ER다이어그램을 통해 시스템에서 사용하는 각 데이터의 상호 관계를 읽어낼 수 있음
- ER다이어그램은 엔티티, 속성, 관계 3가지로 구성

엔티티는 실세계의 객체나 개념을 데이터로 표현한 한 것으로, 데이터베이스 설계나 시스템 분석에서 매우 중요한 개념으로, 데이터 모델링의 기본 단위

엔티티의 정의

- 실세계의 객체: 사람, 장소, 물건, 개념 등과 같이 독립적으로 존재하며 고유한 의미를 가진 것.
- 데이터로 표현: 데이터베이스나 시스템에서 관리해야 할 정보의 단위로 구조화된 형
- 예를 들어, "고객", "주문", "제품" 등은 모두 엔티티가 될 수 있음

엔티티의 특징

- 독립적 존재: 엔티티는 다른 엔티티와 구분되는 고유한 특성을 가짐 (예: " 고객 " 엔티티는 " 주문 " 엔티티와 독립적으로 존재)
- 속성(Attribute)으로 구성: 엔티티는 여러 속성으로 이루어져 있음 (예: "고객" 엔티티는 이름, 전화번호, 주소 등의 속성을 가질 수 있음)
- 식별 가능: 각 엔티티는 고유하게 식별할 수 있는 기본키(Primary Key)를 가짐 (예: 고객 " 엔티티는 고객 ID로 식별)

엔티티와 테이블의 관계

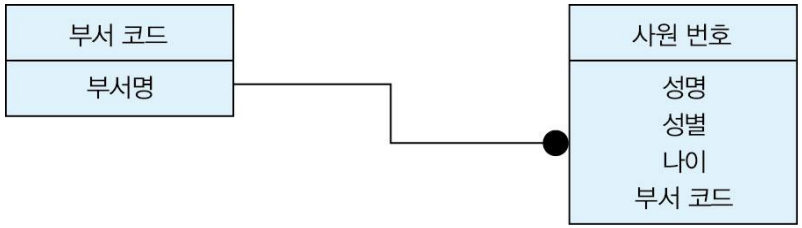
- 데이터베이스에서 엔티티는 테이블(Table)로 표현
- 엔티티의 속성은 테이블의 열(Column)이 됨
- 엔티티의 각 인스턴스(예: 특정 고객)는 테이블의 행(Row)으로 저장

예: 엔티티: "고객"

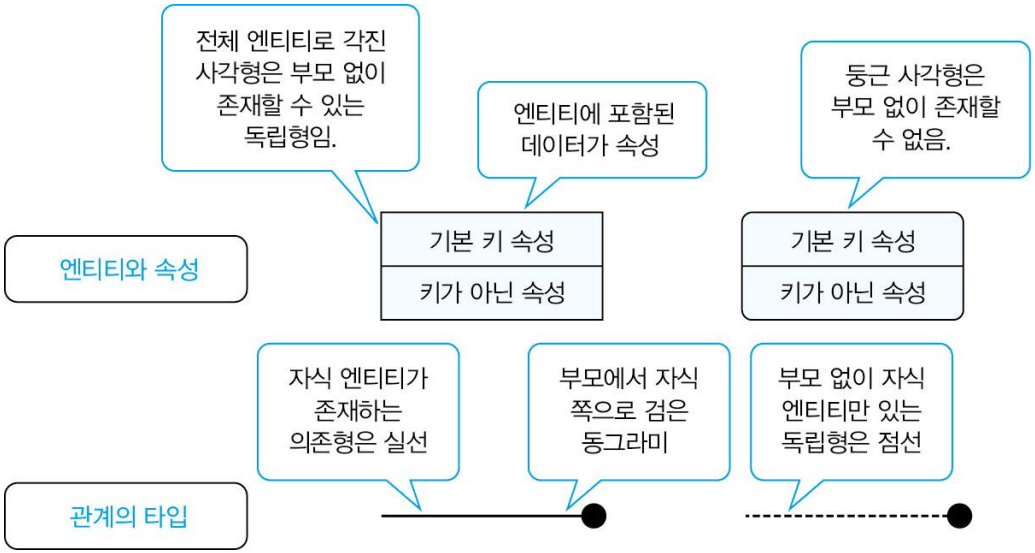
- 테이블: Customer
- 속성(열): CustomerID, Name, Phone, Address
- 행: 고객 1, 고객 2, 고객 3, ...

관계는 몇 가지 종류가 있으며 엔티티 사이의 의존 정도에 따라 점선 또는 직선으로 표현

- 아래 그림과 같이 사원-부서의 관계 표현 가능
- 부서는 부서 코드로 식별하며, 직원은 부서 코드를 가짐으로 소속을 나타내는 관계



엔티티, 속성, 관계



관계의 요소

- 개체(Entity): 관계를 형성하는 주체(예: 학생, 강의, 교수)
- 관계(Relationship): 개체들 간의 연관성(예: "수강한다", "지도한다")
- 카디널리티(Cardinality, 관계 차수): 한 개체가 다른 개체와 맺을 수 있는 관계의 수(1:1, 1:M, M:N)

관계의 종류 (카디널리티)

- 일대일(1:1, One-to-One)
  - 한 개체가 다른 개체의 한 개체와만 관계를 맺는 경우
  - 예: "사용자 - 사용자 프로필" (한 사용자는 하나의 프로필만 가짐)
- 일대다(1:M, One-to-Many)
  - 한 개체가 여러 개의 다른 개체와 관계를 맺을 수 있는 경우
  - 예: "교수 - 강의" (한 교수가 여러 강의를 담당할 수 있음)
- 다대다(M:N, Many-to-Many)
  - 여러 개체가 서로 여러 개의 개체와 관계를 맺는 경우
  - 예: "학생 - 강의" (한 학생이 여러 강의를 수강할 수 있고, 한 강의도 여러 학생이 들을 수 있음)
  - 일반적으로 다대다 관계는 **별도의 관계 테이블(교차 테이블, Join Table)**을 만들어 1:M 관계로 변환

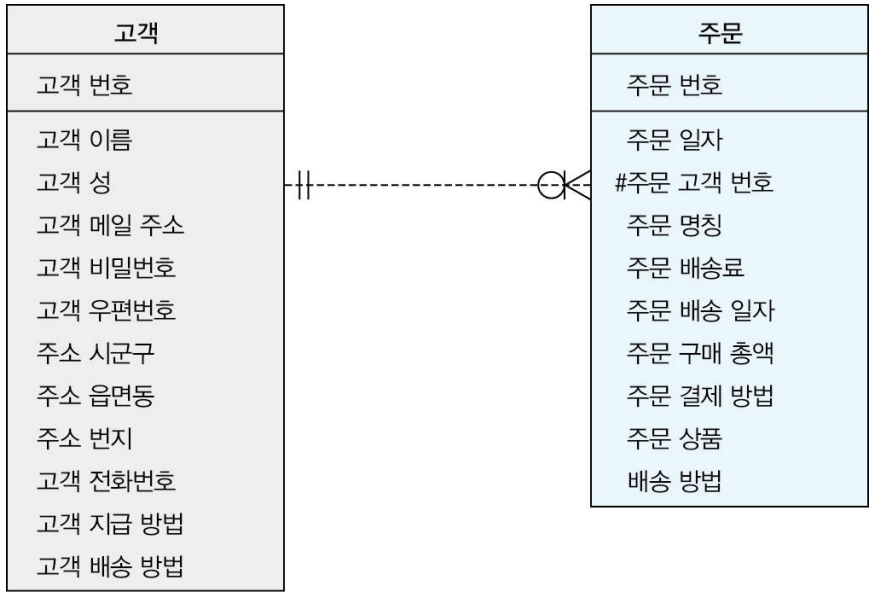
마스터 데이터와 트랜잭션 데이터

엔티티는 특성에 따라 리소스와 이벤트 타입의 엔티티로 분류할 수도 있음

- **리소스 엔티티**: 데이터베이스 설계 시 "마스터"가 될 수 있는 엔티티
  - 조직이나 시스템에서 지속적으로 관리되고 추적되는 핵심 비즈니스 객체
  - 독립적으로 존재하며, 비교적 안정적인 특성
  - 주로 명사형으로 표현
  - 예: 고객, 제품, 직원, 부서, 매장
- **이벤트 엔티티**: 데이터 베이스 설계 시 "트랜잭션" 이 될 수 있는 엔티티
  - 리소스 엔티티 간의 상호작용이나 특정 시점에 발생하는 업무 사건
  - 시간적 특성을 가지며, 언제 발생했는지가 중요
  - 주로 동사의 명사형으로 표현
  - 예: 주문, 입고, 결제, 배송, 예약

마스터 데이터는 기업 조직이 업무 활동을 하기 위한 기반이 되는 데이터이며, 트랜잭션 데이터는 매일 일어나는 일상적인 업무 등 기업 조직의 활동에 의해 발생하는 사건

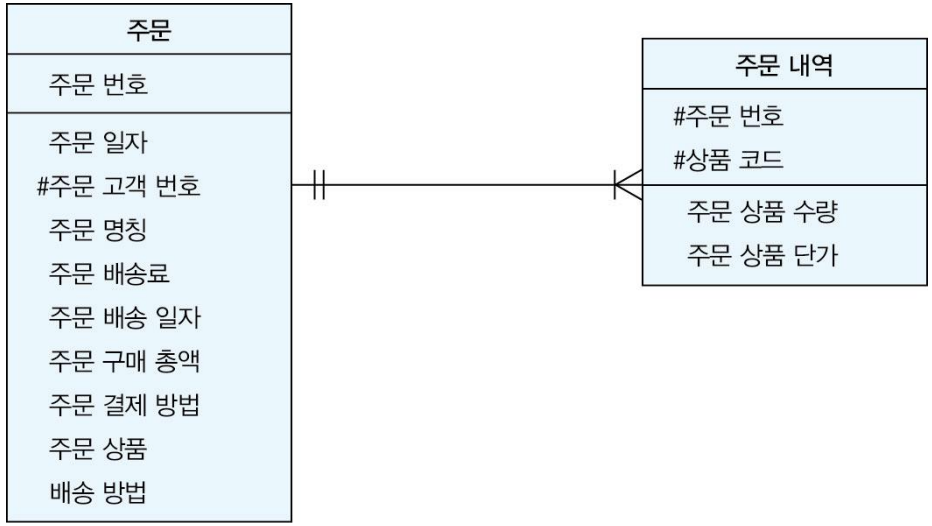
리소스 엔티티와 이벤트 엔티티



옆 그림의 예처럼 주문 엔티티는 다른 엔티티에 의존하지 않기 때문에 독립 엔티티이며, 주문 엔티티 없이 주문 내역 엔티티는 생성되지 않기 때문에 종속 엔티티라고 할 수 있음.

또한 주문과 주문 내역은 종속(식별)관계에 있어서 실선으로 이어져 있고, 나머지 주문은 하나 이상의 "주문 내역" (1대 n)과 연관됨

독립 엔티티와 종속 엔티티의 차이



스텝1: 리소스 엔티티 추출 및 정의

- 업무 규칙, 업무 규정, 요구 사항, 요구 분석 명세서 또는 새로운 시스템 전체를 나타내는 다이어그램으로부터 리소스 엔티티를 추출 후 영역을 정의함
- 모든 엔티티는 표현에 충분히 의미가 있어야 함. 즉 "각 엔티티가 무엇을 위해 존재하는가?", "누가 이 엔티티를 관리할 것인가?"와 같은 질문을 던지면서 체크함
- 현재 잘 알지 못하는 범위라면 설명, 제약, 규칙, 범위가 될 수 있는 것은 빠짐없이 찾아내야 함.
- 엔터프라이즈 시스템에서 이미 마스터 데이터로 관리되고 있는 경우 다른 시스템에서 사용하기 위한 마스터 데이터가 존재하는 경우, 그 마스터 데이터를 그대로 사용 가능함
- 존재하지 않는 경우에는 개념 데이터 설계과정을 통해 향후 마스터로 사용될 모델을 수립함

스텝2: 이벤트 엔티티 추출 및 정의

- 리소스와 마찬가지로 현재 파악 가능한 이벤트 엔티티를 찾아내고, 각 엔티티의 영역을 정의

스텝3: 추출한 엔티티 사이 관계 표현

- 엔티티 사이 관계 의미를 파악하고 정의함. 모든 관계의 선은 방향성에 대한 이유가 있음. 서로의 관계를 명확히 파악하여 의미를 정의해야 함
- 관계에 대한 의미를 정의하는 문장은 "~한다" 는 동사 구문을 갖게 됨
- 화살표 방향에 따라 "부모 엔티티는 자식 엔티티를 ~한다" 로 통일하면 파악하기 쉬움
- 이 작업이 진행되면 엔티티를 나타내는 박스에 하나씩 선으로 연결되고, 이 과정이 완료되면 개념 데이터 모델의 큰 틀이 만들어짐

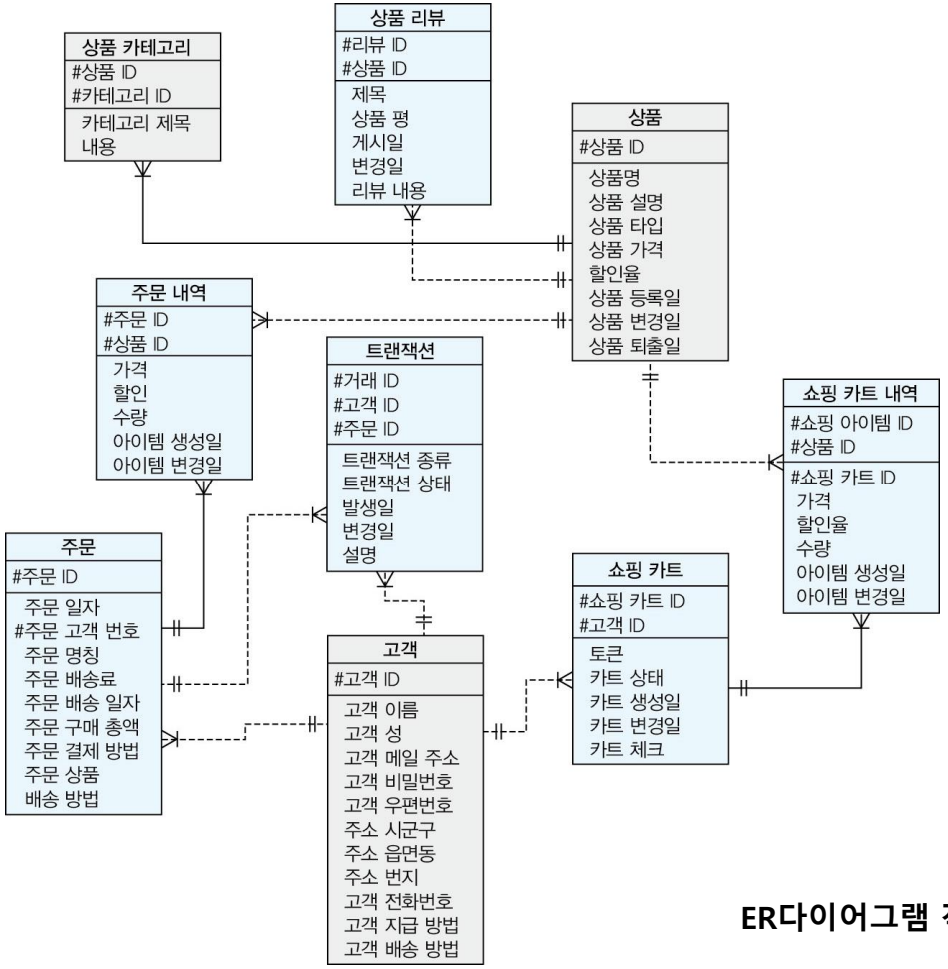
스텝4: 추출한 엔티티 사이 관계 표현

- ER다이어그램에서 각 엔티티의 개요와 관계의 의미가 잘 정의되어 있고, 완벽하게 설명되어 있는지 확인
- 다이어그램의 엔티티를 작성한 후에는 반드시 정의하고 엔티티의 관계를 파악한 후에는 반드시 선을 그어 정의함
- 자세한 정의가 없는 박스나 선만으로는 모델이 완벽하지 않고 구현에 도움되지 않음 → 불완전한 모델을 작성하는 것으로는 대상 업무의 데이터 처리에 결함이 되기 때문

ER다이어그램 검증을 위해 체크할 사항은 다음과 같음

ER다이어그램 체크리스트

대상	체크 항목	질문
엔티티	이름	중복된 것 없나? 변수 스타일의 이름은 피할 것
	필요성	꼭 필요한 것인가? 속성이 되어야 하지 않나?
	일반화	일반화/특수화가 필요한가?
관계	다중도	1:1, 1:n 적합한가? 올바른 방향인가?
	중복	중복된 관계는 없는가?
	이름	이름이 올바로 붙여 있는가? 관계 이름이 중복된 것은 없는가?
완전성	의존 관계	의존 관계와 독립 관계가 구별되어 정확히 표현되었나?
	엔티티	빠진 엔티티는 없는가?
	연관	빠진 관계는 없는가?



ER다이어그램 작성 사례

- **CRUD표는 어떤 작업을 수행할 때 ER다이어그램에 나타난 엔티티에 대해 어떤 조작을 할 것인지 표 형식으로 나타낸 것**
- 각 기능에서 엔티티에 대하여 하는 조작은 다음과 같이 구분
  - 생성 (Create)
  - 참조 (Refer)
  - 갱신 (Update)
  - 삭제 (Delete)
- CRUD 표를 작성하면 엔티티에 대하여 수행할 동작을 분석할 수 있으며 같은 엔티티를 접근하는 기능이 동일한 서브시스템 내에 들어가 있는지 확인할 수 있음

CRUD 표

프로세스 엔티티	주문	고객	상품
상품 분류 관리			RU
상품 옵션 관리			RU
상품 등록			C
상품 목록			RU
상품 정보 일괄 변경			U
삭제 상품 목록			R
상품 진열 관리			RU
상품 정보 표시 설정			RU
상품 조회			R
회원 등록		C	
회원 탈퇴		D	
개인 정보 변경		U	
주문 수집	R		
주문 확정	RU		
취소 주문	D		
반품	RU		

외부 시스템과 데이터를 교환하는 시스템에는 외부 인터페이스 설계가 필수

- 외부 인터페이스 설계는 외부 시스템에 전달할 데이터와 받는 데이터의 명세를 정의
- 일반적으로 외부와 상호 작용하는 파일의 규격을 외부 파일 스펙이라 부르며, 주고받는 데이터의 내용은 데이터 교환 스펙이라고 부름
  - 외부 파일 스펙: 문자 코드, 뉴라인 코드, 필드 구성, 타입, 값이 비어있는 데이터 존재 여부, 최대 레코드 수 등
  - 데이터 교환 스펙: 사용할 네트워크 매체의 종류, 전송 프로토콜, 응용 프로그램 프로토콜, 파일 스펙에 준하여 교환할 데이터 형식 등을 기술

인터페이스 ID	SIF-001
송수신 구분	수신
시스템 이름	온라인 쇼핑 시스템
업무	결제 처리
서비스 이름	신용카드 결제

순번	항목	키	타입	크기	NULL 허용	설명	조건
1	신용카드 번호	Y	varchar	16	N		
2	승인 번호	Y	varchar	8	N		
3	승인 일자		date	8	N		
4	승인 금액		var	15	N		
5	매입 금액		var	15	N		
6	매입 일자		date	8	N		
7	가맹점 사업자 등록 번호		varchar	10	N		
8	가맹점의 업종명		char	10	N	업종 코드 일치	
9	가맹점명		char	20	N		
10	가맹점 대표자명		char	10	Y		
11	가맹점 업종 코드		varchar	3	N	업종 코드 참조	
12	우편번호		char	6	N		
13	거래 구분		varchar	2	N	할부 개월	
14	가맹점 주소 1		char	15	Y		
15	가맹점 주소 2		char	15	Y		

외부 인터페이스 스펙 예

기본 설계 과정에서 실시한 아키텍처, 데이터, 사용자 인터페이스 등의 개별 설계를 마친 후에는 기본 설계서로 정리

- 각 항목의 설명은 시스템 제안서보다 더 넓고 상세해야 함. 따라서 한꺼번에 작성하기는 어려움. 기본 설계서를 통합하기 전에 시스템의 다양한 측면에 대한 개별 설계를 실시하고, 이들을 취합하여 기본 설계서로 정리하게 됨
- 표에서 나타난 것 처럼 개발 팀과 협력할 외부인에게 필요한 사항이 대부분. 개별 설계 과정에서 언급한 것처럼 작성하는 개별 설계 내용은 시스템의 종류와 목적에 따라 달라짐. 따라서 또 다른 개별 설계 내용이 나올 가능성도 있지만 기본적으로 여기에 포함된 내용이 기본 설계 나옴
- 목표, 방침, 개요, 기능에 대한 요구 분석 명세, 시스템 제안서, 업무 흐름, DFD등을 참조하여 상세하게 설명해야 함. 특히 기능은 사용자 인터페이스 항목부터 나오는 각 설명 항목의 요약에 해당하는 것이므로 가장 중요하며, 누락이 발생하지 않도록 주의해야 함

기본 설계서의 항목

기본 설계서		개별 설계		
목적과 정책	목표	요구 분석 명세서 시스템 제안서	업무 흐름도 DFD	
개요	영역 개요			
기능	주요 기능			
UI	입력 기능 조작 방법 결과 표시			화면 레이아웃 인쇄물 레이아웃
시스템 구성	전체 구성			
SW 구성	SW의 구성 기존 SW를 사용하는 경우 구성 내용			코드 정의 데이터베이스 목록 ER 다이어그램 CRUD 표
HW 구성	HW 구성. 사용하는 HW 개요			
네트워크 구성	네트워크 구성 사용하는 네트워크 개요 사용하는 프로그램 개요			
시스템 인터페이스	파일 규격 데이터 교환 스펙			파일 규격 데이터 교환 스펙

- 사용자 인터페이스는 주로 화면 레이아웃과 출력물 레이아웃을 기반으로 작성하며 시스템이 표시하는 화면과 시스템에 입력하는 화면, 출력 화면 등 모든 것을 포함해야 함
- 소프트웨어 구성은 코드 정의, 데이터베이스 목록, 필요한 ER다이어그램, 및 CRUD표를 기반으로 기술함
- 대규모 시스템인 경우 취급하는 데이터의 양이 많아지기 때문에 ER다이어그램도 함께 방대하게 됨. 이런 경우 다이어그램을 업무 영역별로 분리하여 작성하고, 자세한 목표로 정의해야 함
- 외부 시스템 인터페이스는 파일 규격, 데이터 교환 스펙에 따라 외부와 상호 작용하는 정보의 형식을 기술함. 이 때 두 규격은 외부 시스템 사이에 해석 차이로 문제 생기지 않도록 일관되고 누락된 것 없이 완전해야 함
- 시스템 구성, 하드웨어 구성, 네트워크 구성은 요구분석 명세서, 시스템 제안서, 업무 흐름을 바탕으로 요구사항을 충족하도록 기술해야 함 → 기본 설계 단계가 되면 이러한 항목은 각 분야의 전문 엔지니어가 담당하는 것이 일반적이며 소프트웨어 개발자는 이를 잘 이해하고 협력하여 취합하는 것이 바람직

학습정리

- 자료 설계 과정은 코드 설계, 논리 데이터 설계, 물리 데이터 설계, 테이블 정의 순으로 이뤄지며, 자료의 특성에 따라 다양한 방법을 적용할 수 있다.
- ER다이어그램은 논리 데이터 설계의 대표적인 방법으로, 데이터 사이 관계와 데이터베이스 구조를 정의하는 기법이다.
- CRUD매트릭스를 통해 데이터가 실제로 어떻게 다뤄지는지 알 수 있다.
- 개별 설계를 마친 후 이를 취합한 것이 기본 설계서이며, 각 항목을 빠짐없이 포함하는 것이 원칙